COURSE CODE/TITLE:	CPE10- Data Structures and Algorithm	**SECTION:**	CPE21S4
DATE PERFORMED:	7/31/26	**DATE SUBMITTED:**	7/31/26
NAME:	Peñaflorida,Wrio Carlo M.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. HOA4.1 STACKS

1. PROCEDURE OUTPUT

Tasks:

- Modify the code given above to include a function that will display all elements in the stack.
- Provide a description of each operation provided.
- Include your output in the output section of your activity template.

main.cpp

main.cpp > main()

```cpp
#include<iostream>
#include "stackList.h"

int main(){

    std::cout<< " --- PUSHING 'c' 'p' 'e' ONTO THE STACK ---- \n";
    //push
    push('c');
    push('p');
    push('e');

    //display the current top
    Top<char>(); // Top should be 'e'
    displayALL<char>();
    std::cout<<"\n ------- Popping all the elements ------- \n";

    while(!isEmpty<char>()){
        std::cout<< " Popped: "<< pop<char>() <<std::endl;
    }

std::cout<<"\n ------- Testing the Underflow ------- \n";
pop<char>();

return 0;
}
```

stackList.h > pop<T>()

```h
#ifndef STACK_LIST_H
#define STACK_LIST_H

//class node that contains data and the next
template <typename T>
class Node{
public:
T data; // VALUE THAT WE NEED STORE
Node* next; // POINTER TO THE NEXT NODE
};

//global pointer for the top of the stack
template <typename T>
Node<T> *head = nullptr;

//operations for the stack

// push
template <typename T>
void push(T newData){
//create a new node and assign the newData
Node<T>* newNode = new Node<T>;
newNode->data = newData;

//new node points to current top, then becomes the new top
newNode->next = head<T>;
head<T> = newNode;
}

// pop
template <typename T>
T pop(){
//check if the stack is empty:
if(head<T> == nullptr){
std::cout<< " stack overflow. "<<std::endl;
}

//save the current top node
Node<T>* temp = head<T>;
//store its data
T tempVal = temp->data;
// move the head to the next node
head<T> = head<T>->next;
//free the removed node
delete temp;
//return the removed value
return tempVal;
}
```

stackList.h > pop<T>()

```h
#ifndef STACK_LIST_H
#define STACK_LIST_H

//class node that contains data and the next
template <typename T>
class Node{
public:
T data; // VALUE THAT WE NEED STORE
Node* next; // POINTER TO THE NEXT NODE
};

//global pointer for the top of the stack
template <typename T>
Node<T> *head = nullptr;

//operations for the stack

// push
template <typename T>
void push(T newData){
//create a new node and assign the newData
Node<T>* newNode = new Node<T>;
newNode->data = newData;

//new node points to current top, then becomes the new top
newNode->next = head<T>;
head<T> = newNode;
}

// pop
template <typename T>
T pop(){
//check if the stack is empty:
if(head<T> == nullptr){
std::cout<< " stack overflow. "<<std::endl;
}

//save the current top node
Node<T>* temp = head<T>;
//store its data
T tempVal = temp->data;
// move the head to the next node
head<T> = head<T>->next;
//free the removed node
delete temp;
//return the removed value
return tempVal;
}
```

Output:

```
> cd "c:\Users
 --- PUSHING 'c' 'p' 'e' ONTO THE STACK ----
Top of Stack: e
Displaying the stack:
e
p
c


------- Popping all the elements -------
Popped: e
Popped: p
Popped: c

------- Testing the Underflow -------
stack overflow.
PS C:\Users\TIPQC\Documents\CPE10S4\PenafloridaCPE10\HOA4\ILOB2>
```

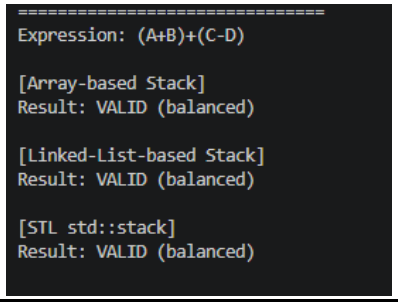
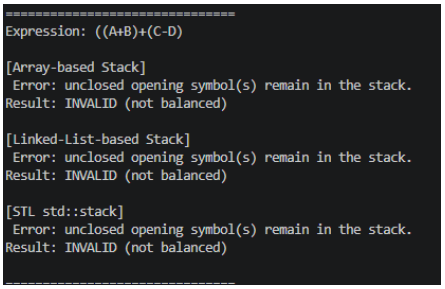
push Adds valon top of the stack
Pop Removes and returns the top element; prints “Stack underflow” and returns a default value if empty.
isEmpty Returns true if the stack has no elements **Top** Prints the top element without removing it
displayALL Prints every element from top to bottom without removing them

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using an implementation of stack:
The following problem definition and algorithm is provided for checking the balancing of symbols. Create an implementation using stacks. Your output must include the following: a. Stack using Arrays
b. Stack using Linked Lists
c. (Optional) Stack using C++ STL Problem Definition:
Stacks can be used to check whether the given expression has balanced symbols. This algorithm is very useful in compilers. Each time the parser reads one character at a time. If the character is an opening delimiter such as (, {, or [- then it is written to the stack. When a closing delimiter is encountered like), }, or]-the stack is popped. The opening and closing delimiters are then compared. If they match, the parsing of the string continues. If they do not match, the parser indicates that there is an error on the line.
Steps:
1. Create a Stack.
2. While(end of input is not reached) {
a) If the character read is not a symbol to be balanced, ignore it.
b) If the character is an opening symbol, push it onto the stack.
c) If it is a closing symbol:
i) Report an error if the stack is empty.
ii) Otherwise, pop the stack.
d) If the symbol popped is not the corresponding opening symbol, report an error.

3. At the end of input, if the stack is not empty: report an error.
Self-Checking:

For the following cases, complete the table using the code you created.

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y		all of it is balanced/valid
((A+B)+(C-D)	N		can't just check for mismatches mid-scan,also need to verify the stack is empty after processing the whole string.

((A+B)+[C-D])	Y	========== Expression: ((A+B)+[C-D]) [Array-based Stack] Result: VALID (balanced) [Linked-List-based Stack] Result: VALID (balanced) [STL std::stack] Result: VALID (balanced) ==========	shows that checker isn't just counting opens/closes, it's actually verifying type-matching on pop
((A+B]+[C-D]}	N	========== Expression: ((A+B]+[C-D]} [Array-based Stack] Error: mismatched symbols '(' and ']'. Result: INVALID (not balanced) [Linked-List-based Stack] Error: mismatched symbols '(' and ']'. Result: INVALID (not balanced) [STL std::stack] Error: mismatched symbols '(' and ']'. Result: INVALID (not balanced) PS C:\Users\wrioc\OneDrive\Desktop\acads\DSA\hoa4\HOA4>	Mismatch caught correctly, all three report "mismatched symbols '(' and ']'"

Tools Analysis:
● How do the different internal representations affect the implementation and usage of the stack? **Array** fixed capacity access via index but risks overflow

LinkedList each push/pop involves heap allocation/deallocation which is slower and has pointer.

STL less code and less control and standard tested.

main.cpp

main.cpp > isMatchingPair(char, char)

```cpp
#include <iostream>
#include <string>
#include <stack> // for the STL version
#include "stack.h" // StackArray + StackList (custom implementations)

bool isMatchingPair(char open, char close) {
if (open == '(' && close == ')') return true;
if (open == '{' && close == '}') return true;
if (open == '[' && close == ']') return true;
return false;
}

/* ---------- Array-based version ---------- */
bool checkBalancedArray(const std::string& expr) {
StackArray<char> s;
for (char ch : expr) {
if (ch == '(' || ch == '{' || ch == '[') {
s.push(ch);
} else if (ch == ')' || ch == '}' || ch == ']') {
if (s.isEmpty()) {
std::cout << " Error: closing symbol '" << ch
<< "' found with no matching opening symbol.\n";
return false;
}
char top = s.pop();
if (!isMatchingPair(top, ch)) {
std::cout << " Error: mismatched symbols '" << top
<< "' and '" << ch << "'.\n";
return false;
}
}
}
if (!s.isEmpty()) {
std::cout << " Error: unclosed opening symbol(s) remain in the stack.\n";
return false;
}
return true;
}

/* ---------- Linked-list-based version ---------- */
bool checkBalancedList(const std::string& expr) {
StackList<char> s;
for (char ch : expr) {
if (ch == '(' || ch == '{' || ch == '[') {
s.push(ch);
} else if (ch == ')' || ch == '}' || ch == ']') {
if (s.isEmpty()) {
std::cout << " Error: closing symbol '" << ch
<< "' found with no matching opening symbol.\n";
return false;
}
char top = s.pop();
```

```cpp
"((A+B)+[C-D])",
"((A+B]+[C-D])"
};

for (const std::string& expr : testCases) {
std::cout << "\n================================\n";
std::cout << "Expression: " << expr << "\n";

std::cout << "\n[Array-based Stack]\n";
bool resA = checkBalancedArray(expr);
std::cout << "Result: " << (resA ? "VALID (balanced)" : "INVALID (not balanced)") << "\n";

std::cout << "\n[Linked-List-based Stack]\n";
bool resL = checkBalancedList(expr);
std::cout << "Result: " << (resL ? "VALID (balanced)" : "INVALID (not balanced)") << "\n";

std::cout << "\n[STL std::stack]\n";
bool resS = checkBalancedSTL(expr);
std::cout << "Result: " << (resS ? "VALID (balanced)" : "INVALID (not balanced)") << "\n";
}

return 0;
}
```



```
52  char top = s.pop();
53  if (!isMatchingPair(top, ch)) {
54  std::cout << " Error: mismatched symbols '" << top
55  << "' and '" << ch << "'.\n";
56  return false;
57  }
58  }
59  }
60  if (!s.isEmpty()) {
61  std::cout << " Error: unclosed opening symbol(s) remain in the stack.\n";
62  return false;
63  }
64  return true;
65  }
66
67  /* ---------- C++ STL std::stack version ---------- */
68  bool checkBalancedSTL(const std::string& expr) {
69  std::stack<char> s;
70  for (char ch : expr) {
71  if (ch == '(' || ch == '{' || ch == '[') {
72  s.push(ch);
73  } else if (ch == ')' || ch == '}' || ch == ']') {
74  if (s.empty()) {
75  std::cout << " Error: closing symbol '" << ch
76  << "' found with no matching opening symbol.\n";
77  return false;
78  }
79  char top = s.top();
80  s.pop();
81  if (!isMatchingPair(top, ch)) {
82  std::cout << " Error: mismatched symbols '" << top
83  << "' and '" << ch << "'.\n";
84  return false;
85  }
86  }
87  }
88  if (!s.empty()) {
89  std::cout << " Error: unclosed opening symbol(s) remain in the stack.\n";
90  return false;
91  }
92  return true;
93  }
94
95  int main() {
96  std::string testCases[] = {
97  "(A+B)+(C-D)",
98  "((A+B)+(C-D)",
99  "((A+B)+[C-D])",
```

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

COMPUTER ENGINEERING

QUEZON CITY

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: gemini

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: for understanding the key concepts of the codes for array

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.